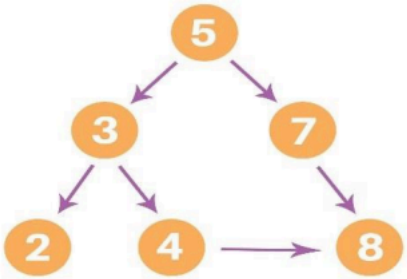


Figure: Graph for BFS Implementation

```
graph = {
  '5' : ['3','7'],
  '3' : ['2', '4'],
  '7' : ['8'],
  '2' : [],
  '4' : ['8'],
  '8' : []
}

visited = []
queue = []

def bfs(start):
    visited.append(start)
    queue.append(start)

    while queue:
        node = queue.pop(0)
        print(node, end=" ")

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

print("BFS Traversal:")
bfs('5')
```

```
BFS Traversal:
5 3 7 2 4 8
```

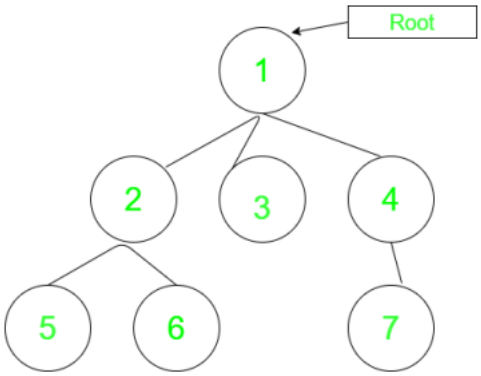


Figure: Tree for BFS Implementation

```
# Tree BFS + utilities

def printParents(node, adj, parent):
    if parent == 0:
        print(node, "-> Root")
    else:
        print(node, "->", parent)
```

```
    for cur in adj[node]:
        if cur != parent:
            printParents(cur, adj, node)


def printChildren(root, adj):
    q = [root]
    visited = [0] * len(adj)

    while q:
        node = q.pop(0)
        visited[node] = 1

        print(node, "-> ", end=" ")
        for cur in adj[node]:
            if visited[cur] == 0:
                print(cur, end=" ")
                q.append(cur)
        print()


def printLeafNodes(root, adj):
    print("Leaf Nodes:")
    for i in range(1, len(adj)):
        if len(adj[i]) == 1 and i != root:
            print(i, end=" ")
    print()


def printDegrees(root, adj):
    print("Degrees:")
    for i in range(1, len(adj)):
        if i == root:
            print(i, ":", len(adj[i]))
        else:
            print(i, ":", len(adj[i]) - 1)


# Create tree
N = 7
root = 1
adj = [[] for _ in range(N + 1)]

# edges
adj[1].append(2); adj[2].append(1)
adj[1].append(3); adj[3].append(1)
adj[1].append(4); adj[4].append(1)
adj[2].append(5); adj[5].append(2)
adj[2].append(6); adj[6].append(2)
adj[4].append(7); adj[7].append(4)

print("Parents:")
printParents(root, adj, 0)

print("\nChildren (BFS):")
printChildren(root, adj)

printLeafNodes(root, adj)

printDegrees(root, adj)
```

```
Parents:
1 -> Root
2 -> 1
5 -> 2
6 -> 2
3 -> 1
4 -> 1
7 -> 4

Children (BFS):
1 ->  2 3 4
2 ->  5 6
3 ->
4 ->  7
5 ->
6 ->
7 ->
Leaf Nodes:
3 5 6 7
Degrees:
1 : 3
2 : 2
3 : 0
```

```
4 : 1
5 : 0
6 : 0
7 : 0
```

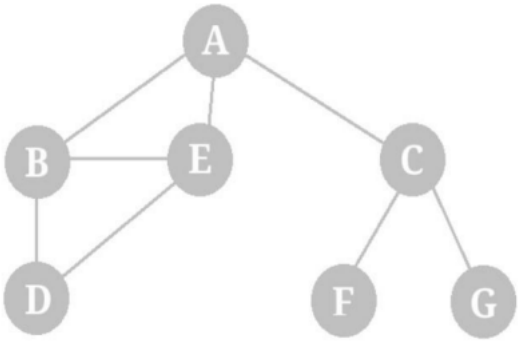


Figure: Graph for Activity 3

```
# BFS on weighted graph (NOT optimal, just for lab)

graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('D', 7), ('E', 3)],
    'C': [('F', 5)],
    'D': [('G', 1)],
    'E': [('G', 6)],
    'F': [('G', 2)],
    'G': []
}

from collections import deque

def bfs_cost(start, goal):
    queue = deque()
    queue.append((start, [start], 0))  # node, path, cost

    visited = set()

    while queue:
        node, path, cost = queue.popleft()

        if node == goal:
            print("Path:", " -> ".join(path))
            print("Total Cost:", cost)
            return

        visited.add(node)

        for neighbor, weight in graph[node]:
            if neighbor not in visited:
                queue.append((neighbor, path + [neighbor], cost + weight))

print("BFS Path with Cost:")
bfs_cost('A', 'G')
```

```
BFS Path with Cost:
Path: A -> B -> D -> G
Total Cost: 10
```

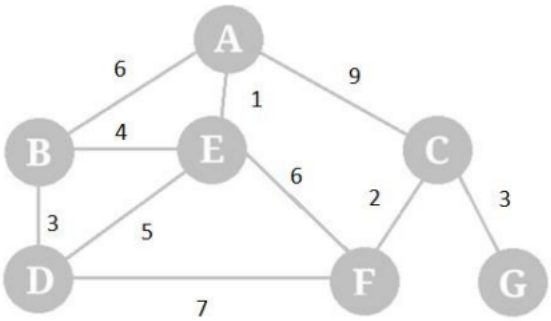
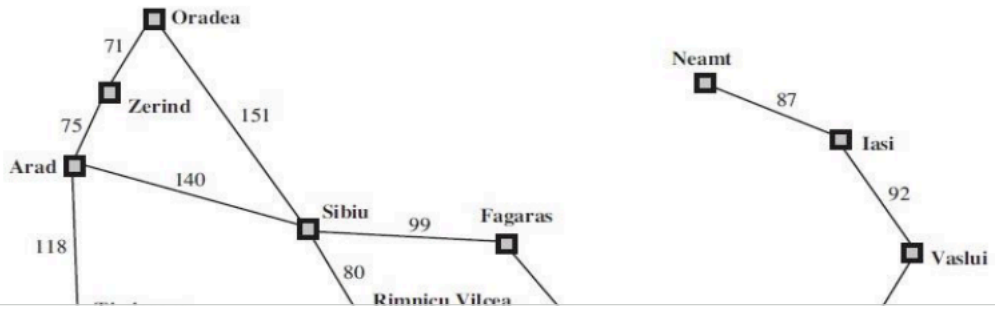


Figure: Weighted Graph for Activity Lab Task 1

```
# Romania Map BFS (Arad to Bucharest)

graph = {
    'Arad': [('Sibiu', 140), ('Timisoara', 118), ('Zerind', 75)],
    'Sibiu': [('Fagaras', 99), ('Rimnicu', 80)],
    'Fagaras': [('Bucharest', 211)],
    'Rimnicu': [('Pitesti', 97)],
    'Pitesti': [('Bucharest', 101)],
    'Timisoara': [('Lugoj', 111)],
    'Lugoj': [('Mehadia', 70)],
    'Mehadia': [('Drobeta', 75)],
    'Drobeta': [('Craiova', 120)],
    'Craiova': [('Pitesti', 138)],
    'Zerind': [('Oradea', 71)],
    'Oradea': [('Sibiu', 151)],
    'Bucharest': []
}

from collections import deque

def bfs_romania(start, goal):
    queue = deque()
    queue.append((start, [start], 0))

    visited = set()

    while queue:
        node, path, cost = queue.popleft()

        if node == goal:
            print("Path:", " -> ".join(path))
            print("Total Distance:", cost)
            return

        visited.add(node)

        for neighbor, dist in graph.get(node, []):
            if neighbor not in visited:
                queue.append((neighbor, path + [neighbor], cost + dist))

print("BFS from Arad to Bucharest:")
bfs_romania('Arad', 'Bucharest')
```

```
BFS from Arad to Bucharest:
Path: Arad -> Sibiu -> Fagaras -> Bucharest
Total Distance: 450
```




```
from collections import deque

graph = {
    'Arad': [('Zerind', 75), ('Timisoara', 118), ('Sibiu', 140)],
    'Zerind': [('Arad', 75), ('Oradea', 71)],
    'Oradea': [('Zerind', 71), ('Sibiu', 151)],
    'Timisoara': [('Arad', 118), ('Lugoj', 111)],
    'Lugoj': [('Timisoara', 111), ('Mehadia', 70)],
    'Mehadia': [('Lugoj', 70), ('Drobeta', 75)],
    'Drobeta': [('Mehadia', 75), ('Craiova', 120)],
    'Craiova': [('Drobeta', 120), ('Pitesti', 138), ('Rimnicu', 146)],
    'Sibiu': [('Arad', 140), ('Oradea', 151), ('Fagaras', 99), ('Rimnicu', 80)],
    'Rimnicu': [('Sibiu', 80), ('Craiova', 146), ('Pitesti', 97)],
    'Fagaras': [('Sibiu', 99), ('Bucharest', 211)],
    'Pitesti': [('Rimnicu', 97), ('Craiova', 138), ('Bucharest', 101)],
    'Bucharest': [('Fagaras', 211), ('Pitesti', 101)]
}

def bfs(start, goal):
    queue = deque()
    queue.append((start, [start], 0))  # node, path, cost
    visited = set()

    while queue:
        node, path, cost = queue.popleft()

        if node == goal:
            print("Path:", " -> ".join(path))
            print("Total Distance:", cost)
            return

        if node not in visited:
            visited.add(node)

            for neighbor, distance in graph[node]:
                queue.append((neighbor, path + [neighbor], cost + distance))


print("BFS from Arad to Bucharest:")
bfs('Arad', 'Bucharest')
```

```
BFS from Arad to Bucharest:
Path: Arad -> Sibiu -> Fagaras -> Bucharest
Total Distance: 450
```

Double-click (or enter) to edit

Double-click (or enter) to edit